SCRIPTS AND FUNCTIONS

Variables and Assignments

- MATLAB uses a default variable named *ans* if an expression is typed at the command prompt and it is not assigned to a variable. For example, the result of the expression

```
>> 5* 4
ans = 20
```

Variables and Assignments…

Choosing a variable name rules

(1) The name must begin with a letter of the alphabet. After that, the name can contain letters, digits, and the underscore character (e.g., value_1), but it cannot have a space.

(2) MATLAB is case-sensitive, which means there is a difference between upper- and lowercase letters. So, variables called mynum , MYNUM , and Mynum are all different.

Variables and Assignments…

(3) Underscores are valid in variable names but cannot begin with the name .For example,
_r is not a valid variable name in MatLab,
r_ is a valid variable name.
To test if a variable name is valid in MatLab , use the comand ***isvarname*** to test it

For example,
isvarname _r

isvarname r_

Variables and Assignments…

(4) Avoid using reserved names as variables in MatLab e.g. function, if, else, sin, cos. Although some names of predefined functions such as (sin, cos, tan etc) could be used as variable names, avoid them at all cost as they may cause problems.

Variables and Assignments…

Initializing :

- Most at times, the values of variables change. Putting the first or initial value in a variable is called initializing the variable

For example: myVar = 3;

This means that the variable myVar has been initialized to 3. We can perform some operations on the variable.

myVar = myVar +myVar

Response myVar = 6

Variables and Assignments…

Increment and decrement :

- Adding to a variable is called incrementing.

For example

myVar= myVar + 1

This means myVar has been increased my a value of 1

In a similar fashion, subtracting will result in decrement

myVar = myVar-1

Expressions and Operators

- Expressions can be created using values, variables, operators, parentheses and built-in functions.

  For example;

  my_exp = 2 *cos(pi/2)

  To suppress an expression use a semicolon

  i.e my_exp=2 *cos(pi/2); will be suppressed

Expressions and Operators…

Here are some of the common operators that can be used with numerical expressions:

+ addition

- negation, subtraction

* Multiplication

/ division (divided by, e.g., 10/5 is 2)

\ division (divided into, e.g., 5\10 is 2)

^ exponentiation (e.g., 5^2 is 25)

Expressions and Operators…

- **Operators precedence rules**

Some operators have precedence over others. For example, in the expression

6+ 5 * 3, the multiplication takes precedence over the addition, so first 5 is

multiplied by 3, then 6 is added to the result. 6+5*3 = 21

Using parentheses can change the precedence in an expression: (6+5)*3=33

Expressions and Operators…

- **Operators precedence rules**…

The following is the precedence (from highest to lowest):

( )	parentheses
^	exponentiation
-	negation
*, /, \	all multiplication and division
+, -	addition and subtraction

Expressions and Operators…

- **Operators precedence rules**…

Examples:

Evaluate the following:

(1) 4^2-3
(2) 4^(2-3)
(3) 4 *2 - 9/3

(1)Multiply (3)Subtract (2) Divide

SCRIPTS

- Sequence of Matlab commands stored in M-file and saved
- The contents of a script can be displayed in the Command Window using the *type* command
- The script can be executed, or run, by simply entering the name of the file (without the .m extension).

SCRIPTS…

- To create a script, click on File, then New, then Script. A new window called Editor will appear
- To create a new script, simply type the sequence of statements. Save it as M-file

Examples

```
% This script calculates area of a circle
clear all;
close;
rad = 6
area =pi*(rad^2)
```

Save this as script1.m

SCRIPTS…

Running a script

- To a MatLab script, point to the directory where the script is saved. To view the content use the ***type*** command and the script name without the extension .m
- In our example; type script1
- There are two ways to view a script once it has been written: either open the Editor Window to view it, or use the type command as shown here to display it in the Command Window.

SCRIPTS…

- To actually run or execute the script, the name of the file is entered at the prompt (again, without the .m). For example

```
>> script1
radius =5
area =113.0973
```

SCRIPTS…

Clear and Close All

It is always advisable to use the following two commands:

clear;

close all;

The clear command clears all variables from Matlab's memory and makes sure that you don't have leftover (junk) in Matlab that will interfere with your code. The close all command closes any figure windows that are open.

Debugging Code

- Any error in a computer program is called a bug .
- It is very common to have a bug in your code. This could be syntax error, runtime error or logical error.
- Syntax errors are mistakes in using the language. Examples of syntax errors are missing a comma or a quotation mark, or misspelling a word.

Debugging Code…

- Runtime, or execution-time, errors are found when a script or function is executing. With most languages, an example of a runtime error would be attempting to divide by zero.
- A logical error is a mistake in reasoning by the programmer, but it is not a mistake in the programming language.

Make Your Code Readable

In any case try to make your code readable by humans including yourself.

Document your scripts by including lines in it that begin with %, like this:

```
% This is a comment
```

If you get an error running your code, comment that part that gives you an error. Run the code again to find out if the problem is solved

FUNCTIONS

- MatLab has rich collections of in-build functions for trigonometric, logarithmic, and hyperbolic functions etc.
- MatLab has two types of M-files: script files and function files. A script file contains sequence of MatLab commands saved as M-files. A function file contains set of repeated commands.

FUNCTIONS…

function definition

function [output variable(s)] = function_name(inputs)

- The function header (the first line) contains the reserved word *function*
- Since the function returns a result, the name of the output argument is followed by the assignment operator .

FUNCTIONS…

- The name of the function, IMPORTANT; should be the same as the name of the M-file in which this function is stored to avoid confusion.
- The input arguments in parentheses, which correspond to the arguments are passed to the function in the function call
- The body of the function, which includes all statements and eventually must put a value in the output argument. *end* at the end of the function.

FUNCTIONS…

Examples:

(1)

```
% This function add two numbers

function a = add( b, c)
a = b+c;
end
```

(2)

The function called calcarea calculates and returns the area of a circle. It is stored in a file called calcarea.m .

```
% calcarea calculates the area of a circle
function area = calcarea(rad)
area = pi * rad.^2;
end
```

FUNCTIONS…

Calling a function:

A function is called by its name followed by parenthesis.
For example; to call the function add( ).
We then supply the arguments of the function, so the call function will be like
add(5,6)
The output will be 11

To call **calarea** function
calarea(3), where 3 is the radius i.e. argument

FUNCTIONS…

Functions can take **more than one argument**, as has been the case of add( ). All the arguments must be supplied to avoid error.

For example, the volume of a cone is given by $V = \frac{1}{3}\pi r^2 h$

where r is the radius of the circular base and h

is the height of the cone. A function that calculates the

volume of a cone needs both the radius and the height:

M-file Functions

- A function is different than a script in that the input parameters it needs are passed to it with argument lists like Matlab commands

For example: Code the expression as m-file function

$$f(x,y) = \frac{\sin(xy)}{x^2 + y^2}$$

Solution:

```
function f = tri(x,y)
f = sin(x*y)./(x.^2 +y.^2);
end
```

M-file Functions…

The first line of the function file is of the form

function output=name(input)

The word *function* is required, output is what the function passes back to whomever called it, name should match the filename of the script (e.g. “tri.m”) as above, and input is the argument list of the function.

Calling a function

- When a function is called, the arguments passed in must match in number and type defined in the definition. The function m-file needs to assign the output an appropriate value.
- The names of the variables in input and output of a function are local to the function, so you don't have to worry about overwriting variables in the file that called the function.

Calling a function…

- If you want variables used outside the function to be visible inside the function and vice-versa, use Matlab's global command. For instance, if you put the command

  global a b c;

Functions With Multiple Outputs

- Some functions can return more than one output. When a function returns more than one output variable, the names of these results are put in square brackets.

Syntax:

function [output1,output2]= f_name(input)

When you call the function, you use the same type of format to receive the output

Functions With Multiple Outputs…

Example

% Calculating energy and wave function

```
function [x,psi,E] = Square(n,a,NPoints)
xmin=0;
xmax =a;
x = linspace(xmin,xmax,NPoints);
k=n*pi/a;
psi=sqrt(2/a)*sin(k*x);
hbar=1.05e-34;
m=9.11e-31;
E=n^2*pi^2*hbar^2/(2*m*(a*1e9)^2.
```

Continued…

You can call the square function in another script like this

```
clear; close all;
% remember that n must be a positive integer
 n=3;
% Set the width
a=0.1;
% Get the values
[x,psi,Energy] = Square(n,a,100);
% Make the plot and label it
plot(x,psi)
title(s);
xlabel(‘x (nm)); ylabel('\Psi(x)');
```

Continued…

Note that we didn't have to call the variables the same names when we used them outside the function in the main script (e.g. we used Energy instead of E). This is because all variables inside functions are local to these programs and Matlab doesn't even know about them in the command window.